

4 Documentation_Kubernetes

Kubernetes est un système open-source permettant d'automatiser le déploiement, la mise à l'échelle et la gestion des applications conteneurisées. Les conteneurs qui composent une application sont regroupés dans des unités logiques pour en faciliter la gestion et la découverte. Kubernetes s'appuie sur 15 années d'expérience dans la gestion de charges de travail de production (workloads) chez Google, associé aux meilleurs idées et pratiques de la communauté.

Sommaire :

- [1 - Configuration de 3 VMs, Installation de K3S et Création du Cluster](#)
 - [2 - Déploiement des services \(Nginx, Apache, Mysql, Mariadb\) avec YAML](#)
 - [3 - Test de Haute Disponibilité \(HA\)](#)
 - [4 - Stockage Persistant](#)
 - [5 - ConfigMaps](#)
 - [6 - Mode "Secret"](#)
 - [7 - Règles RBAC](#)
 - [8 - Helm](#)
 - [9 - Docker, Docker Swarm & Kubernetes](#)
 - [10 - Conclusion et compétences développées sur le projet](#)
-

1 - Configuration de 3 VMs, Installation de K3S et Création du Cluster

On configure 3 machines virtuelles Debian sans interface graphique, chacune avec une IP statique, avant d'installer K3S et de les assembler en cluster avec un master et deux workers.

1.1 Configuration des adresses IP statiques

On configure une IP fixe sur chaque VM dans `/etc/network/interfaces` pour éviter que les adresses changent au redémarrage, ce qui est indispensable pour la communication stable entre les nœuds du cluster.

```
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group de
fault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 00:0c:29:82:dc:ef brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2982dcef
    inet 192.168.163.131/24 brd 192.168.163.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::cf0e:989b:f2c6:61b2/64 scope link tentative
        valid_lft forever preferred_lft forever
root@kubes01:~# ping google .com
ping: .com: Nom ou service inconnu
root@kubes01:~# ping google.com
PING google.com (142.251.209.206) 56(84) bytes of data.
64 bytes from ncmrsa-am-in-f14.1e100.net (142.251.209.206): icmp_seq=1 ttl=1
28 time=5.08 ms
64 bytes from ncmrsa-am-in-f14.1e100.net (142.251.209.206): icmp_seq=2 ttl=1
28 time=9.93 ms
64 bytes from ncmrsa-am-in-f14.1e100.net (142.251.209.206): icmp_seq=3 ttl=1
28 time=8.94 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 5.084/7.984/9.928/2.089 ms
root@kubes01:~#
```

```

ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group de
fault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 00:0c:29:60:2b:90 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29602b90
    inet 192.168.163.132/24 brd 192.168.163.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::1749:a361:3fe9:afed/64 scope link tentative
        valid_lft forever preferred_lft forever
root@kubes-02:~# ping google.com
PING google.com (142.251.209.206) 56(84) bytes of data.
64 bytes from ncmrsa-am-in-f14.1e100.net (142.251.209.206): icmp_seq=1 ttl=1
28 time=3.05 ms
64 bytes from ncmrsa-am-in-f14.1e100.net (142.251.209.206): icmp_seq=2 ttl=1
28 time=2.89 ms
64 bytes from ncmrsa-am-in-f14.1e100.net (142.251.209.206): icmp_seq=3 ttl=1
28 time=10.2 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.890/5.368/10.161/3.389 ms
root@kubes-02:~#

```

```

root@kubes03:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group de
fault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 00:0c:29:1d:de:97 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c291dde97
    inet 192.168.163.133/24 brd 192.168.163.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.163.135/24 brd 192.168.163.255 scope global secondary dynam
ic noprefixroute ens33
        valid_lft 1526sec preferred_lft 1301sec
    inet6 fe80::e664:9003:e64e:e438/64 scope link
        valid_lft forever preferred_lft forever
root@kubes03:~# ping google.com
PING google.com (172.217.18.110) 56(84) bytes of data.
64 bytes from ncmrsb-aj-in-f14.1e100.net (172.217.18.110): icmp_seq=1 ttl=12
8 time=3.49 ms
64 bytes from ncmrsb-aj-in-f14.1e100.net (172.217.18.110): icmp_seq=2 ttl=12
8 time=6.01 ms
64 bytes from ncmrsb-aj-in-f14.1e100.net (172.217.18.110): icmp_seq=3 ttl=12
8 time=4.67 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 3.489/4.721/6.008/1.029 ms
root@kubes03:~#

```

1.2 Configuration du fichier /etc/hosts

Le fichier /etc/hosts permet d'associer à une adresse IP un nom de domaine. Ainsi nous pouvons exécuter la commande ping kubes-02.local.

```

root@kubes01:~# cat >> /etc/hosts << 'EOF'
192.168.163.131 kubes-01.local kubes-01
192.168.163.132 kubes-02.local kubes-02
192.168.163.133 kubes-03.local kubes-03
EOF
root@kubes01:~# |

```

```
root@kubes01:~# ping -c 2 kubes-02.local
ping -c 2 kubes-03.local
PING kubes-02.local (192.168.163.132) 56(84) bytes of data.
64 bytes from kubes-02.local (192.168.163.132): icmp_seq=1 ttl=64 time=28.3
ms
64 bytes from kubes-02.local (192.168.163.132): icmp_seq=2 ttl=64 time=1.03
ms

--- kubes-02.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.026/14.678/28.331/13.652 ms
PING kubes-03.local (192.168.163.133) 56(84) bytes of data.
64 bytes from kubes-03.local (192.168.163.133): icmp_seq=1 ttl=64 time=2.40
ms
64 bytes from kubes-03.local (192.168.163.133): icmp_seq=2 ttl=64 time=1.16
ms

--- kubes-03.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.162/1.780/2.399/0.618 ms
root@kubes01:~#
```

1.3 Installation de K3S

K3S (Rancher / SUSE) est une distribution Kubernetes certifiée packagée en un seul binaire de 100 Mo par opposition à K8S. On l'installe sur chaque VM avec une seule commande curl.

```

root@kubes01:~# curl -sL https://get.k3s.io | sh -
[INFO] Finding release for channel stable
[INFO] Using v1.35.5+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.35.5%2Bk3s1/sha256sum-amd64.txt
[INFO] Skipping binary downloaded, installed k3s matches hash
[INFO] Skipping installation of SELinux RPM
[INFO] Skipping /usr/local/bin/kubectl symlink to k3s, already exists
[INFO] Skipping /usr/local/bin/crictl symlink to k3s, already exists
[INFO] Skipping /usr/local/bin/ctr symlink to k3s, already exists
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink '/etc/systemd/system/multi-user.target.wants/k3s.service' ->
'/etc/systemd/system/k3s.service'.
[INFO] No change detected so skipping service start
root@kubes01:~# systemctl status k3s
● k3s.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s.service; enabled; preset: enab>
   Active: active (running) since Tue 2026-06-09 09:39:24 CEST; 14s ago
 Invocation: 8eaf70e5ee1247a7bb066c3086c30dc9
    Docs: https://k3s.io
   Main PID: 11223 (k3s-server)
     Tasks: 46
  Memory: 637.6M (peak: 651.1M)
     CPU: 14.882s
    CGroup: /system.slice/k3s.service
            └─11223 "/usr/local/bin/k3s server"
              └─11283 "containerd "
                └─11972 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b2>
                  └─11978 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b2>

juin 09 09:39:34 kubes01 k3s[11223]: time="2026-06-09T09:39:34+02:00" level>
juin 09 09:39:34 kubes01 k3s[11223]: time="2026-06-09T09:39:34+02:00" level>
juin 09 09:39:34 kubes01 k3s[11223]: time="2026-06-09T09:39:34+02:00" level>
juin 09 09:39:34 kubes01 k3s[11223]: time="2026-06-09T09:39:34+02:00" level>
juin 09 09:39:34 kubes01 k3s[11223]: time="2026-06-09T09:39:34+02:00" level>
juin 09 09:39:34 kubes01 k3s[11223]: I0609 09:39:34.712819 11223 kuberunt>
juin 09 09:39:34 kubes01 k3s[11223]: I0609 09:39:34.714845 11223 kubelet>
juin 09 09:39:36 kubes01 k3s[11223]: time="2026-06-09T09:39:36+02:00" level>
juin 09 09:39:36 kubes01 k3s[11223]: I0609 09:39:36.068361 11223 network_>
juin 09 09:39:36 kubes01 k3s[11223]: I0609 09:39:36.154417 11223 network_>

```

1.4 Récupération du Token sur kubes01 notre master

Le token est une clé d'authentification générée par le master, nécessaire pour autoriser les workers à rejoindre le cluster.

```

root@kubes01:~# cat /var/lib/rancher/k3s/server/node-token
K10cf22cedb6ce7fc63139a1ae366fb2b833cd96c5d452571308e3227b7254771aa::server:
75a12c3ed78e6141540d09f8c799f86f
root@kubes01:~#

```

1.5 Installation de K3S en mode agent sur nos deux autres VMs à partir du Token Master

On installe K3S en mode agent sur kubes-02 et kubes-03 en fournissant l'URL du master et le token récupéré précédemment.

```

root@kubes-02:~# curl -sL https://get.k3s.io | K3S_URL=https://192.168.163.131:6443 K3S_TOKEN="K10cf22cedb6ce7fc63139a1ae366fb2b833cd96c5d452571308e3227b7254771aa::server:75a12c3ed78e6141540d09f8c799f86f" sh -
[INFO] Finding release for channel stable
[INFO] Using v1.35.5+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.35.5%2Bk3s1/sha256sum-amd64.txt
[INFO] Skipping binary downloaded, installed k3s matches hash
[INFO] Skipping installation of SELinux RPM
[INFO] Skipping /usr/local/bin/kubectyl symlink to k3s, already exists
[INFO] Skipping /usr/local/bin/crictl symlink to k3s, already exists
[INFO] Skipping /usr/local/bin/ctr symlink to k3s, already exists
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink '/etc/systemd/system/multi-user.target.wants/k3s-agent.service' -> '/etc/systemd/system/k3s-agent.service'.
[INFO] No change detected so skipping service start
root@kubes-02:~# systemctl start k3s-agent
systemctl status k3s-agent
● k3s-agent.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s-agent.service; enabled; preset=)
   Active: active (running) since Tue 2026-06-09 09:46:12 CEST; 20ms ago
 Invocation: 03f902aec72346fbae107e247ce8605a
    Docs: https://k3s.io
  Process: 6107 ExecStartPre=/sbin/modprobe br_netfilter (code=exited, status=0)
  Process: 6108 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0)
 Main PID: 6110 (k3s-agent)
    Tasks: 36
   Memory: 80.7M (peak: 88.2M)
      CPU: 1.788s
   CGroup: /system.slice/k3s-agent.service
           └─6110 "/usr/local/bin/k3s agent"
             └─6127 "containerd "
               └─6548 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23>
                 └─6734 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23>

juin 09 09:46:12 kubes-02.local k3s[6110]: I0609 09:46:12.484771      6110 ku>
juin 09 09:46:12 kubes-02.local k3s[6110]: I0609 09:46:12.514232      6110 ku>

```

```

root@kubes03:~# curl -sL https://get.k3s.io | K3S_URL=https://192.168.163.1
31:6443 K3S_TOKEN="K10cf22cedb6ce7fc63139a1ae366fb2b833cd96c5d452571308e3227
b7254771aa::server:75a12c3ed78e6141540d09f8c799f86f" sh -
[INFO] Finding release for channel stable
[INFO] Using v1.35.5+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.
35.5%2Bk3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v
1.35.5%2Bk3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubectl symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service
.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink '/etc/systemd/system/multi-user.target.wants/k3s-agent.servi
ce' -> '/etc/systemd/system/k3s-agent.service'.
[INFO] Host iptables-save/iptables-restore tools not found
[INFO] Host ip6tables-save/ip6tables-restore tools not found
[INFO] systemd: Starting k3s-agent
root@kubes03:~# systemctl start k3s-agent
systemctl status k3s-agent
● k3s-agent.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s-agent.service; enabled; preset>
   Active: active (running) since Tue 2026-06-09 09:47:37 CEST; 1s ago
  Invocation: b2f2ada2829749118f16187fd9d1511f
     Docs: https://k3s.io
   Process: 5016 ExecStartPre=/sbin/modprobe br_netfilter (code=exited, st>
   Process: 5017 ExecStartPre=/sbin/modprobe overlay (code=exited, status=>
  Main PID: 5019 (k3s-agent)
     Tasks: 22
    Memory: 316.7M (peak: 358.5M)
       CPU: 3.393s
    CGroup: /system.slice/k3s-agent.service
            └─5019 "/usr/local/bin/k3s agent"
              └─5074 "containerd "

juin 09 09:47:36 kubes03 k3s[5019]: I0609 09:47:36.718229    5019 shared_in>
juin 09 09:47:36 kubes03 k3s[5019]: I0609 09:47:36.719059    5019 iptables.>

```

1.6 Vérification de notre Cluster

On vérifie depuis le master que les 3 nœuds sont bien Ready avec `kubectl get nodes`.

```

root@kubes01:~# kubectl get nodes
NAME                STATUS    ROLES                  AGE     VERSION
kubes-01.local      Ready     control-plane          90s     v1.35.5+k3s1
kubes-02.local      Ready     <none>                  5m3s    v1.35.5+k3s1
kubes-03.local      Ready     <none>                  9s       v1.35.5+k3s1
root@kubes01:~#

```

Les 3 VMs sont configurées, K3S est installé et le cluster est opérationnel avec 1 master et 2 workers. Nous pouvons maintenant déployer nos premières applications sur le cluster.

[Sommaire :](#)

2 - Déploiement des services (Nginx, Apache, Mysql, Mariadb) avec YAML

On déploie les applications conteneurisées sur le cluster K3S en utilisant des fichiers YAML qui décrivent l'état souhaité de chaque application — Kubernetes se charge ensuite de les faire tourner sur les nœuds disponibles

2.1 Edition du fichier YAML

On crée un fichier apps.yaml contenant la définition de chaque application un Deployment qui décrit le conteneur et ses paramètres, et un Service de type NodePort qui expose l'application sur le réseau.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  type: NodePort
  ports:
  - port: 80
    targetPort: 80
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      labels:
        app: apache
    spec:
      containers:
      - name: apache
        image: httpd:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: apache-service
spec:
  selector:
    app: apache
  type: NodePort
  ports:
  - port: 80
    targetPort: 80
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mariadb
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
      - name: mariadb
        image: mariadb:latest
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: "rootpassword"
        ports:
        - containerPort: 3306
---
apiVersion: v1
kind: Service
metadata:
  name: mariadb-service
spec:
  selector:
    app: mariadb
  type: NodePort
  ports:
  - port: 3306
    targetPort: 3306

```

2.2 Lancemenet des services

On applique le fichier YAML sur le cluster avec `kubectl apply -f apps.yaml` Kubernetes crée automatiquement les Deployments et Services pour nginx, apache et mariadb.

```
root@kubes-01:~# kubectl apply -f apps.yaml
deployment.apps/nginx created
service/nginx-service created
deployment.apps/apache created
service/apache-service created
deployment.apps/mariadb created
service/mariadb-service created
root@kubes-01:~#
```

2.3 Vérification de l'état des services

On vérifie que tous les pods sont bien en état Running et que les services sont correctement exposés avec `kubectl get pods` et `kubectl get services`.

```
root@kubes-01:~# kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES
apache-84647dcc49-9rq52             1/1      Running   0           12m   10.42.1.5     kubes-02.local   <none>            <none>
mariadb-c68cf7b8c-28sgs             1/1      Running   0           12m   10.42.1.4     kubes-02.local   <none>            <none>
mariadb-c68cf7b8c-zmtpw             1/1      Running   0           12m   10.42.3.4     kubes-01.local   <none>            <none>
nginx-59f86b59ff-rmd6b             1/1      Running   0           12m   10.42.0.3     kubes-03.local   <none>            <none>
root@kubes-01:~#
```

Les 3 applications sont déployées et accessibles sur le cluster. Nous allons maintenant tester la robustesse de notre infrastructure en simulant une panne d'un nœud.

[Sommaire :](#)

3 - Test de Haute Disponibilité (HA)

La haute disponibilité garantit que les applications restent accessibles même en cas de panne d'un nœud. On teste ce comportement en arrêtant volontairement un worker et en observant Kubernetes redéployer automatiquement les pods sur les nœuds restants.

3.1 Vérification du status des services sur le master

On vérifie l'état initial des pods et leur répartition sur les nœuds avant de simuler une panne, avec `kubectl get pods -o wide`.

```
root@kubes-01:~# kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP            NOD
E                                  NOMINATED NODE   READINESS GATES
apache-84647dcc49-9rq52             1/1      Running   0           50m   10.42.1.5     kub
es-02.local   <none>            <none>
mariadb-c68cf7b8c-28sgs             1/1      Running   0           50m   10.42.1.4     kub
es-02.local   <none>            <none>
mariadb-c68cf7b8c-zmtpw             1/1      Running   0           50m   10.42.3.4     kub
es-01.local   <none>            <none>
nginx-59f86b59ff-rmd6b             1/1      Running   0           50m   10.42.0.3     kub
es-03.local   <none>            <none>
root@kubes-01:~#
```

3.2 Arrêt de l'agent k3s-agent sur kubes-03

On simule une panne en arrêtant volontairement le service k3s-agent sur kubes-03 avec `systemctl stop k3s-agent`.

```
root@kubes-03:~# systemctl stop k3s-agent
root@kubes-03:~#
```

3.3 Revérification du statut des services sur le master

On vérifie que kubes-03 passe bien en état NotReady Kubernetes détecte la panne et va automatiquement redéployer les pods orphelins sur les nœuds disponibles.

```
root@kubes-01:~# kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
kubes-01.local      Ready     control-plane   129m   v1.35.5+k3s1
kubes-02.local      Ready     <none>         133m   v1.35.5+k3s1
kubes-03.local      NotReady  <none>         128m   v1.35.5+k3s1
root@kubes-01:~#
```

3.4 Redéploiement de Nginx sur kubes-02 automatiquement

Kubernetes redéploie automatiquement le pod nginx sur kubes-02 l'ancien pod passe en Terminating et un nouveau pod est créé en Running sur un nœud disponible, prouvant que la HA fonctionne.

```
Toutes les 2,0s: kubectl get pods -o wide
kubes-01.local: Tue Jun 9 12:06:40 2026
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
apache-84647dcc49-9rq52	1/1	Running	0	60m	10.42.1.5	kubes-02.local	<none>	<none>
mariadb-c68cf7b8c-28sgs	1/1	Running	0	60m	10.42.1.4	kubes-02.local	<none>	<none>
mariadb-c68cf7b8c-zmtpw	1/1	Running	0	60m	10.42.3.4	kubes-01.local	<none>	<none>
nginx-59f86b59ff-rmd6b	1/1	Terminating	0	60m	10.42.0.3	kubes-03.local	<none>	<none>
nginx-59f86b59ff-wtwd5	1/1	Running	0	2m45s	10.42.1.6	kubes-02.local	<none>	<none>

La haute disponibilité est validée Kubernetes a automatiquement redéployé nginx sur un nœud disponible sans intervention manuelle. Notre cluster est résilient aux pannes. Nous allons maintenant mettre en place le stockage persistant pour garantir la conservation des données.

[Sommaire :](#)

4 - Stockage Persistant

Les volumes persistants permettent aux applications de conserver leurs données même après le redémarrage ou le redéploiement d'un pod. On utilise NFS pour partager le stockage entre les 3 nœuds du cluster, garantissant que les données restent accessibles peu importe sur quel nœud le pod tourne.

4.1 Installation de NFS sur le serveur (master)

On installe le serveur NFS sur kubes-01 qui va servir de serveur de fichiers partagé pour tout le cluster.

```
root@kubes-01:~# apt install -y nfs-kernel-server
nfs-kernel-server est déjà la version la plus récente (1:2.8.3-1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@kubes-01:~#
```

4.2 Création des dossiers de stockage et attribution des droits

On crée les dossiers /data/nginx et /data/mariadb qui seront partagés, et on leur attribue les droits nécessaires avec chmod 777.

```
root@kubes-01:~# mkdir -p /data/nginx /data/mariadb
root@kubes-01:~# chmod 777 /data/nginx /data/mariadb
root@kubes-01:~#
```

4.3 Configuration de /etc/exports

On configure le fichier /etc/exports pour définir quels dossiers sont partagés et avec quelles permissions pour tout le réseau 192.168.163.0/24.

```
GNU nano 8.4 /etc/exports *
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients. See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/data/nginx 192.168.163.0/24(rw,sync,no_subtree_check,no_root_squash)
/data/mariadb 192.168.163.0/24(rw,sync,no_subtree_check,no_root_squash)
```

4.4 Lancement de l'export

On lance l'export des dossiers partagés avec exportfs -a pour rendre les partages actifs.

```
root@kubes-01:~# exportfs -a
root@kubes-01:~#
```

4.5 Redémarrage de NFS

On redémarre le service NFS avec systemctl restart nfs-kernel-server pour appliquer la configuration.

```
root@kubes-01:~# systemctl restart nfs-kernel-server
root@kubes-01:~#
```

4.6 Vérification du montage de l'export

On vérifie que les deux dossiers sont bien exportés et accessibles sur le réseau avec `exportfs -v`.

```
root@kubes-01:~# exportfs -v
/data/nginx      192.168.163.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_squash)
/data/mariadb    192.168.163.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_squash)
root@kubes-01:~#
```

4.7 Installation de NFS sur les deux VMs workers

On installe le client NFS sur kubes-02 et kubes-03 avec `apt install nfs-common` pour qu'ils puissent accéder aux partages du master.

```
root@kubes-02:~# apt install -y nfs-common
nfs-common est déjà la version la plus récente (1:2.8.3-1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 65
root@kubes-02:~#
```

```
root@kubes-03:~# apt install -y nfs-common
nfs-common est déjà la version la plus récente (1:2.8.3-1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 65
root@kubes-03:~#
```

4.8 Montage de l'export

On monte temporairement le partage NFS sur `/mnt` depuis kubes-02 pour vérifier la connectivité avec le serveur.

```
root@kubes-02:~# mount -t nfs 192.168.163.131:/data/nginx /mnt
```

4.9 Test de l'export

On crée un fichier test sur kubes-01 et on vérifie qu'il est visible depuis kubes-02 ce qui confirme que le partage NFS fonctionne correctement dans les deux sens.

```
root@kubes-01:~# echo "Hello depuis NFS" > /data/nginx/test.html
root@kubes-01:~# |
```

```
root@kubes-02:~# cat /mnt/test.html
Hello depuis NFS
root@kubes-02:~#
```

4.10 Modification du fichier YAML

On ajoute les PersistentVolumes, PersistentVolumeClaims et les volumeMounts dans le fichier apps.yaml pour que nginx et mariadb utilisent le stockage NFS.

```
# AVANT
containers:
- name: mariadb
  image: mariadb:latest
  env:
  - name: MYSQL_ROOT_PASSWORD
    value: "rootpassword"
  ports:
  - containerPort: 3306

# APRÈS
containers:
- name: mariadb
  image: mariadb:latest
  env:
  - name: MYSQL_ROOT_PASSWORD
    value: "rootpassword"
  ports:
  - containerPort: 3306
  volumeMounts:                                # ← AJOUTÉ
  - name: mariadb-storage
    mountPath: /var/lib/mysql
volumes:                                       # ← AJOUTÉ
- name: mariadb-storage
  persistentVolumeClaim:
    claimName: mariadb-pvc
```

4.11 Vérification de nos demandes de stockages persistants et de nos stockages persistants

On vérifie que les PV et PVC sont bien en état Bound avec `kubectl get pv` et `kubectl get pvc`, confirmant que le stockage NFS est correctement lié aux applications.

```

root@kubes-01:~# kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM          STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
mariadb-pv    2Gi       RWX           Retain          Bound   default/mariadb-pvc  default-storageclass  <unset>              9s
nginx-pv      1Gi       RWX           Retain          Bound   default/nginx-pvc    default-storageclass  <unset>              10s
root@kubes-01:~# kubectl get pvc
NAME          STATUS  VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
mariadb-pvc   Bound   mariadb-pv      2Gi       RWX           default-storageclass  <unset>                 17s
nginx-pvc     Bound   nginx-pv        1Gi       RWX           default-storageclass  <unset>                 17s
root@kubes-01:~# kubectl get pods
NAME          READY  STATUS   RESTARTS  AGE
apache-84647dcc49-fdzxv  1/1    Running  0          25s
mariadb-64ddff58f5-br6d5  1/1    Running  0          25s
mariadb-64ddff58f5-dhfb4  1/1    Running  0          25s
nginx-bf54745fd-tz9hp     1/1    Running  0          25s
root@kubes-01:~#

```

Le stockage persistant est en place via NFS les données de nginx et mariadb survivent désormais aux redémarrages des pods. Nous allons maintenant externaliser la configuration des applications avec les ConfigMaps

[Sommaire :](#)

5 - ConfigMaps

Un ConfigMap permet d'externaliser la configuration d'une application en dehors du Deployment, ce qui facilite la gestion et la modification des paramètres sans avoir à redéployer l'application.

5.1 Création de configmap.yaml

On crée un fichier ConfigMap contenant les 4 variables de configuration de MariaDB.

```

GNU nano 8.4                                confi
apiVersion: v1
kind: ConfigMap
metadata:
  name: mariadb-config
data:
  MYSQL_DATABASE: "mabase"
  MYSQL_USER: "monuser"
  MYSQL_PASSWORD: "monpassword"
  MYSQL_ROOT_PASSWORD: "rootpassword"

```

5.2 Application de configmap

On applique le ConfigMap sur le cluster.

```

root@kubes-01:~# kubectl apply -f configmap.yaml
configmap/mariadb-config created
root@kubes-01:~#

```

5.3 Vérification du configmap

On vérifie que le ConfigMap est bien créé avec ses 4 clés.

```
root@kubes-01:~# kubectl get configmap
NAME          DATA      AGE
kube-root-ca.crt  1         5h32m
mariadb-config  4         30s
root@kubes-01:~#
```

5.4 Modification du apps.yaml

On remplace les valeurs codées en dur dans la section `env` de MariaDB par des références au ConfigMap via `configMapKeyRef`.

```
env:
- name: MYSQL_ROOT_PASSWORD
  value: "rootpassword"
```

```
env:
- name: MYSQL_ROOT_PASSWORD
  valueFrom:
    configMapKeyRef:
      name: mariadb-config
      key: MYSQL_ROOT_PASSWORD
- name: MYSQL_DATABASE
  valueFrom:
    configMapKeyRef:
      name: mariadb-config
      key: MYSQL_DATABASE
- name: MYSQL_USER
  valueFrom:
    configMapKeyRef:
      name: mariadb-config
      key: MYSQL_USER
- name: MYSQL_PASSWORD
  valueFrom:
    configMapKeyRef:
      name: mariadb-config
      key: MYSQL_PASSWORD
```

5.5 Application de apps.yaml modifié

On applique le fichier modifié

```

root@kubes-01:~# kubectl apply -f apps.yaml
kubectl get pods
persistentvolume/nginx-pv configured
persistentvolumeclaim/nginx-pvc unchanged
persistentvolume/mariadb-pv configured
persistentvolumeclaim/mariadb-pvc unchanged
deployment.apps/nginx unchanged
service/nginx-service unchanged
deployment.apps/apache unchanged
service/apache-service unchanged
deployment.apps/mariadb configured
service/mariadb-service unchanged
NAME                                READY    STATUS                RESTARTS    AGE
apache-84647dcc49-fdzxv             1/1      Running               0            92m
mariadb-64ddff58f5-br6d5            1/1      Running               0            92m
mariadb-64ddff58f5-dhfb4            0/1      ImagePullBackOff      9 (65m ago)  92m
nginx-bf54745fd-tz9hp               1/1      Running               0            92m
root@kubes-01:~# |

```

5.6 Preuve d'utilisation du configmap

La section Environment du pod confirme que les 4 variables sont lues depuis mariadb-config et non codées en dur.

```

Environment:
  MYSQL_ROOT_PASSWORD: <set to the key 'MYSQL_ROOT_PASSWORD' of config map 'mariadb-config'> Optional: false
  MYSQL_DATABASE:      <set to the key 'MYSQL_DATABASE' of config map 'mariadb-config'> Optional: false
  MYSQL_USER:          <set to the key 'MYSQL_USER' of config map 'mariadb-config'> Optional: false
  MYSQL_PASSWORD:      <set to the key 'MYSQL_PASSWORD' of config map 'mariadb-config'> Optional: false
Mounts:

```

La configuration de MariaDB est externalisée dans un ConfigMap les paramètres sont désormais modifiables sans toucher au Deployment. Nous allons maintenant sécuriser les données sensibles avec les Secrets.

[Sommaire :](#)

6 - Mode "Secret"

Un Secret c'est comme un ConfigMap mais pour les données sensibles : mots de passe, tokens, clés API. La différence principale est que les valeurs sont encodées en Base64 et Kubernetes les protège mieux.

6.1 Création de secret.yaml

On crée le fichier secret.yaml contenant les 4 variables sensibles de MariaDB encodées en Base64, les valeurs ne sont pas lisibles directement contrairement au ConfigMap.

```

GNU nano 8.4                                secret.yaml *
apiVersion: v1
kind: Secret
metadata:
  name: mariadb-secret
type: Opaque
data:
  MYSQL_ROOT_PASSWORD: cm9vdHBhc3N3b3Jk
  MYSQL_PASSWORD: bW9ucGFzc3dvcmQ=
  MYSQL_USER: bW9udXNlcg==
  MYSQL_DATABASE: bWFiYXNl

```

6.2 Application de secret.yaml et vérification

On applique le Secret sur le cluster et on vérifie sa création Kubernetes affiche 4 données de type Opaque, sans jamais révéler les valeurs encodées.

```

root@kubes-01:~# kubectl apply -f secret.yaml
kubectl get secret
secret/mariadb-secret created
NAME                TYPE         DATA   AGE
mariadb-secret      Opaque       4        0s
root@kubes-01:~#

```

6.3 Mise à jour de apps.yaml

On remplace les références configMapKeyRef par des secretKeyRef dans la section env de MariaDB pour que le pod lise ses variables depuis le Secret.

```
env:
- name: MYSQL_ROOT_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mariadb-secret
      key: MYSQL_ROOT_PASSWORD
- name: MYSQL_DATABASE
  valueFrom:
    secretKeyRef:
      name: mariadb-secret
      key: MYSQL_DATABASE
- name: MYSQL_USER
  valueFrom:
    secretKeyRef:
      name: mariadb-secret
      key: MYSQL_USER
- name: MYSQL_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mariadb-secret
      key: MYSQL_PASSWORD
ports:
- containerPort: 3306
volumeMounts:
- name: mariadb-storage
  mountPath: /var/lib/mysql
```

6.4 Application de apps.yaml

On applique le fichier mis à jour deployment.apps/mariadb configured confirme que MariaDB utilise désormais le Secret pour sa configuration.


```

root@kubes-01:~# nano apps.yaml
root@kubes-01:~# kubectl apply --dry-run=client -f apps.yaml
kubectl apply -f apps.yaml
kubectl get pods
persistentvolume/nginx-pv configured (dry run)
persistentvolumeclaim/nginx-pvc configured (dry run)
persistentvolume/mariadb-pv configured (dry run)
persistentvolumeclaim/mariadb-pvc configured (dry run)
deployment.apps/nginx configured (dry run)
service/nginx-service configured (dry run)
deployment.apps/apache configured (dry run)
service/apache-service configured (dry run)
deployment.apps/mariadb configured (dry run)
service/mariadb-service configured (dry run)
persistentvolume/nginx-pv configured
persistentvolumeclaim/nginx-pvc unchanged
persistentvolume/mariadb-pv configured
persistentvolumeclaim/mariadb-pvc unchanged
deployment.apps/nginx unchanged
service/nginx-service unchanged
deployment.apps/apache unchanged
service/apache-service unchanged
deployment.apps/mariadb configured
service/mariadb-service unchanged
NAME                                READY    STATUS    RESTARTS    AGE
apache-84647dcc49-fdzxv             1/1      Running   1 (18h ago)  20h
mariadb-656c875bd8-qtvdq            1/1      Running   0            18h
nginx-bf54745fd-tz9hp               1/1      Running   1 (18h ago)  20h
root@kubes-01:~#

```

6.5 Vérification des variables

La section Environment du pod confirme que les 4 variables sont lues depuis le Secret mariadb-secret et non depuis le ConfigMap ou codées en dur.

```

Environment:
  MYSQL_ROOT_PASSWORD: <set to the key 'MYSQL_ROOT_PASSWORD' in secret 'mariadb-secret'> Optional: false
  MYSQL_DATABASE:      <set to the key 'MYSQL_DATABASE' in secret 'mariadb-secret'> Optional: false
  MYSQL_USER:          <set to the key 'MYSQL_USER' in secret 'mariadb-secret'> Optional: false
  MYSQL_PASSWORD:      <set to the key 'MYSQL_PASSWORD' in secret 'mariadb-secret'> Optional: false

```

Les données sensibles de MariaDB sont maintenant sécurisées dans un Secret Kubernetes. Nous allons maintenant mettre en place les règles RBAC pour contrôler les accès au cluster.

[Sommaire :](#)

7 - Règles RBAC

RBAC = **R**ole **B**ased **A**ccess **C**ontrol = Contrôle d'accès basé sur les rôles. C'est le système de permissions de Kubernetes, il définit qui peut faire quoi sur le cluster.

7.1 Création de rbac.yaml

On crée le fichier rbac.yaml contenant les 3 ressources nécessaires : un ServiceAccount qui représente l'identité, un Role qui définit les permissions, et un RoleBinding qui les associe.

```
GNU nano 8.4 rbac.yaml *
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitoring-user
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: monitoring-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: monitoring-user
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

7.2 Application

On applique le fichier sur le cluster, les 3 ressources monitoring-user, pod-reader et monitoring-binding sont créées en une seule commande.

```
root@kubes-01:~# kubectl apply -f rbac.yaml
serviceaccount/monitoring-user created
role.rbac.authorization.k8s.io/pod-reader created
rolebinding.rbac.authorization.k8s.io/monitoring-binding created
root@kubes-01:~#
```

7.3 Vérification du ServiceAccount

On vérifie que le ServiceAccount `monitoring-user` est bien créé dans le namespace `default` aux côtés du ServiceAccount `system`.

```
root@kubes-01:~# kubectl get serviceaccount
NAME                AGE
default              24h
monitoring-user      27s
root@kubes-01:~#
```

7.4 Vérification du rôle

On vérifie que le Role `pod-reader` autorise uniquement les actions `get`, `list` et `watch` sur les pods, aucune autre ressource ni action n'est permise.

```
root@kubes-01:~# kubectl get role
kubectl describe role pod-reader
NAME                CREATED AT
pod-reader           2026-06-10T08:09:22Z
Name:                pod-reader
Labels:               <none>
Annotations:          <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  pods       []                  []              [get list watch]
root@kubes-01:~#
```

7.5 Vérification du RoleBinding

On vérifie que le RoleBinding `monitoring-binding` lie bien le Role `pod-reader` au ServiceAccount `monitoring-user` dans le namespace `default`.

```

root@kubes-01:~# kubectl get role
kubectl describe role pod-reader
NAME                CREATED AT
pod-reader          2026-06-10T08:09:22Z
Name:               pod-reader
Labels:             <none>
Annotations:        <none>
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----      -
  pods          []                 []              [get list watch]
root@kubes-01:~# kubectl get rolebinding
kubectl describe rolebinding monitoring-binding
NAME                ROLE                AGE
monitoring-binding  Role/pod-reader     87s
Name:               monitoring-binding
Labels:             <none>
Annotations:        <none>
Role:
  Kind:  Role
  Name:  pod-reader
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount  monitoring-user  default
root@kubes-01:~#

```

Les règles RBAC sont en place , les accès au cluster sont maintenant contrôlés par des rôles. Nous allons maintenant installer Helm pour simplifier le déploiement d'applications sur Kubernetes.

[Sommaire :](#)

8 - Helm

Helm est le gestionnaire de paquets de Kubernetes, comme apt pour Debian. Il permet de déployer, mettre à jour et désinstaller des applications via des **Charts** préconfigurés, sans avoir à écrire manuellement des fichiers YAML complexes.

8.1 Installation de Helm sur kubes-01

On installe Helm v3 sur le master en une seule commande curl, Helm est installé dans /usr/local/bin/helm et immédiatement disponible.

```

root@kubes-01:~# curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent
                                Dload  Upload  Total  Spent    Left  Spee
d
  0      0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
100 11929 100 11929    0     0  199k      0 --:--:-- --:--:-- --:--:-- 20
0k
[WARNING] Could not find git. It is required for plugin installation.
Downloading https://get.helm.sh/helm-v3.21.0-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
root@kubes-01:~# helm version
version.BuildInfo{Version:"v3.21.0", GitCommit:"e0878d41b711792be60777fd65ad
23a101e6b85f", GitTreeState:"clean", GoVersion:"go1.25.10"}
root@kubes-01:~#

```

8.2 Repository Helm

On ajoute le repository Bitnami qui contient des centaines d'applications prêtes à déployer, puis on met à jour la liste des Charts disponibles.

```

root@kubes-01:~# helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
"bitnami" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
Update Complete. ✨Happy Helming!✨
root@kubes-01:~#

```

8.3 Installation d'une application via Helm (nginx)

On déploie nginx en une seule commande helm install Helm crée automatiquement tous les Deployments, Services et autres ressources nécessaires.

```

root@kubes-01:~# export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
helm install mon-nginx bitnami/nginx
NAME: mon-nginx
LAST DEPLOYED: Wed Jun 10 10:19:44 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.3
APP VERSION: 1.31.1

```

8.4 Vérification de la release

On vérifie avec helm list que la release mon-nginx est bien déployée en REVISION 1, et avec kubectl get pods que le pod est Running.

```

root@kubes-01:~# helm list
kubectl get pods
NAME                NAMESPACE      REVISION      UPDATED
NAME                STATUS          CHART          APP VERSION
mon-nginx           default         1              2026-06-10 10:19:44.96365541
4 +0200 CEST        deployed        nginx-25.0.3   1.31.1
NAME                READY   STATUS    RESTARTS   AGE
apache-84647dcc49-fdzxv 1/1     Running   1 (18h ago) 20h
mariadb-68c58ddc8b-5tns6 1/1     Running   0           20m
mon-nginx-77fd75878f-5t85n 1/1     Running   0           2m3s
nginx-bf54745fd-tz9hp 1/1     Running   1 (18h ago) 20h
root@kubes-01:~#

```

8.5 Mise à jour d'une release

On met à jour la release avec `helm upgrade` en passant `replicaCount=2` Helm passe automatiquement en REVISION 2 et crée un second pod nginx.

```

root@kubes-01:~# helm upgrade mon-nginx bitnami/nginx --set replicaCount=2
Release "mon-nginx" has been upgraded. Happy Helming!
NAME: mon-nginx
LAST DEPLOYED: Wed Jun 10 10:24:09 2026
NAMESPACE: default
STATUS: deployed
REVISION: 2
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.3
APP VERSION: 1.31.1

```

8.6 Désinstallation d'une release

On désinstalle toute l'application avec `helm uninstall` toutes les ressources créées par Helm sont supprimées en une seule commande.

```

root@kubes-01:~# helm uninstall mon-nginx
kubectl get pods
release "mon-nginx" uninstalled
NAME                READY   STATUS    RESTARTS   AGE
apache-84647dcc49-fdzxv 1/1     Running   1 (18h ago) 20h
mariadb-68c58ddc8b-5tns6 1/1     Running   0           24m
mon-nginx-77fd75878f-5t85n 0/1     Completed 0           5m44s
mon-nginx-77fd75878f-8rql9 0/1     Completed 0           80s
nginx-bf54745fd-tz9hp 1/1     Running   1 (18h ago) 20h
root@kubes-01:~#

```

Helm simplifie considérablement la gestion des applications sur Kubernetes. Le projet est maintenant complet nous avons mis en place un cluster K3S fonctionnel avec toutes les fonctionnalités clés de Kubernetes.

[Sommaire :](#)

9 - Docker, Docker Swarm & Kubernetes

Pourquoi utiliser Kubernetes plutôt que Docker ou Docker Swarm, et dans quels cas chaque outil est-il le plus adapté ?

- Docker est la brique de base, il permet de faire tourner des conteneurs sur une seule machine. C'est l'outil idéal pour le développement local et les tests, mais il ne gère pas la distribution sur plusieurs serveurs.
- Docker Swarm est la solution d'orchestration native de Docker. Il est particulièrement adapté aux infrastructures classiques on-premise, c'est-à-dire des serveurs physiques dans une salle machine locale. Sa simplicité de configuration en fait un bon choix pour des petites équipes qui veulent de la haute disponibilité sans la complexité de Kubernetes. Si ton infrastructure tient sur quelques serveurs dans un datacenter local, Docker Swarm est suffisant et facile à maintenir.
- Kubernetes en revanche a été conçu dès le départ pour le **cloud**. Créé par Google, il est nativement intégré aux grands fournisseurs cloud comme AWS, Google Cloud et Azure. Il excelle dans les environnements où les ressources sont dynamiques, on peut ajouter ou supprimer des nœuds à la volée, scaler automatiquement selon la charge, et déployer sur plusieurs régions géographiques simultanément. **K3S** est la version allégée de Kubernetes, parfaite pour apprendre ou déployer sur des infrastructures plus modestes tout en gardant la puissance de Kubernetes.

Le choix entre ces outils dépend donc directement de l'infrastructure cible, Docker Swarm reste pertinent pour les entreprises avec des serveurs on-premise, tandis que Kubernetes s'impose comme le standard incontournable du cloud moderne.

[Sommaire :](#)

10 - Conclusion et compétences développées sur le projet

Ce projet nous a permis de mettre en place une infrastructure Kubernetes complète, de la configuration des VMs jusqu'au déploiement d'applications en haute disponibilité. Nous avons parcouru l'ensemble des fonctionnalités clés de Kubernetes orchestration, stockage persistant, gestion de la configuration, sécurité et déploiement simplifié avec Helm.

Compétences développées sur le projet :

Administration système

- Configuration de machines virtuelles Debian sans interface graphique
- Gestion des interfaces réseau et des adresses IP statiques
- Mise en place d'un serveur NFS pour le stockage partagé

Virtualisation et conteneurisation

- Installation et configuration d'un cluster K3S
- Déploiement d'applications conteneurisées avec Kubernetes
- Gestion de la haute disponibilité et du redéploiement automatique

Sécurité des infrastructures

- Externalisation de la configuration avec les ConfigMaps
- Sécurisation des données sensibles avec les Secrets Kubernetes
- Mise en place des règles RBAC pour contrôler les accès au cluster

Supervision et gestion

- Utilisation de kubectl pour superviser l'état du cluster
- Analyse des logs et diagnostic des erreurs sur les pods
- Gestion du cycle de vie des applications avec Helm

[Sommaire :](#)
